

GUIA DE MIGRACION Y ACTUALIZACION EN EL TRABAJO

Sistema de Gestion de Reclamos | Actualizacion sin perdida de datos historicos

GARANTIA DE CONSERVACION DE DATOS:

Siguiendo estos pasos, todas las tablas existentes (reclamos, usuarios, sucursales y auditorias) se mantendran 100% intactas. Los volúmenes de la base de datos no se eliminan.

PASO 1: Respaldo preventivo de la Base de Datos (Opcional por seguridad)

Antes de iniciar cualquier cambio, genere una copia de seguridad en archivo SQL de la base de datos actual en su computadora del trabajo:

```
docker exec -t reclamos_postgres pg_dump -U postgres -d reclamos_db > backup_previo.sql
```

** Importante: Este comando crea un archivo de respaldo local sin detener el servicio.*

PASO 2: Descargar los ultimos cambios del repositorio (Git)

Obtenga el código actualizado que ya fue subido a GitHub (incluye la nueva gestión de tareas, columna de estado, corrección del filtro de asesores y frases de perfil):

```
git pull origin main
```

** Importante: Ejecute este comando ubicado en la carpeta principal del proyecto en su PC de trabajo.*

PASO 3: Aplicar migraciones a la Base de Datos existente (Sin tocar los datos actuales)

Agregue la nueva columna en la tabla usuarios y cree la nueva tabla tareas_asignadas directamente en PostgreSQL:

```
docker exec -i reclamos_postgres psql -U postgres -d reclamos_db -c "  
ALTER TABLE usuarios ADD COLUMN IF NOT EXISTS descripcion VARCHAR(255) DEFAULT '';  
CREATE TABLE IF NOT EXISTS tareas_asignadas (  
  id SERIAL PRIMARY KEY,  
  usuario_id INT NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,  
  creado_por_id INT NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,  
  sucursal_1_id INT NOT NULL REFERENCES sucursales(id) ON DELETE RESTRICT,  
  sucursal_2_id INT REFERENCES sucursales(id) ON DELETE SET NULL,  
  tarea VARCHAR(50) NOT NULL,  
  completada BOOLEAN DEFAULT FALSE,  
  completada_at TIMESTAMP WITH TIME ZONE,  
  activo BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);"
```

** Importante: Este comando es idempotente (IF NOT EXISTS): si una columna o tabla ya existe, no causara error ni modificara registros existentes.*

GUIA DE MIGRACION Y ACTUALIZACION (CONTINUACION)

Sistema de Gestion de Reclamos | Actualizacion sin perdida de datos historicos

PASO 4: Reconstruir unicamente el contenedor de la aplicacion Node.js

Reconstruya la imagen de la app para que tome los nuevos controladores, rutas y estilos. El contenedor de base de datos reclamos_postgres continua activo y sin tocar:

```
docker compose up -d --build app
```

** Importante: No use docker compose down -v, ya que -v borraría los volúmenes. Usar solo el comando indicado arriba.*

PASO 5: Verificación final en el navegador web

Abra el sistema en Google Chrome, Firefox o Edge y fuerce la recarga limpia para evitar que el navegador use archivos CSS o JS en cache:

Presione: Ctrl + F5 (Recarga forzada)

** Importante: Compruebe que la aplicación responda en <http://localhost:3000>*

CHECKLIST DE FUNCIONALIDADES NUEVAS A COMPROBAR:

- [v] Gestion de Reclamos: Distribucion en dos columnas (60% Formulario / 40% Mis Tareas Asignadas).
- [v] Discriminacion de Tareas: Sucursales fijas del día y Tareas diarias con casilla interactiva de completado.
- [v] Tablero de Asignar Tareas (Admin / Supervisor): Disposicion al 50%/50% con columna Estado (Marcada / Pendiente).
- [v] Historial: Filtro de Asesores desbloqueado para todos los roles (Asesor, Supervisor, Espectador, Admin).
- [v] Perfil de Usuario: Frase o descripción personal de hasta 150 caracteres visible en modales.
- [v] Unicidad: Maximo un reclamo por abonado, sucursal y fecha.
- [v] Auditoria: Tareas y modificaciones registradas con usuario, fecha y detalle.